



MIDTERM PROJECT REPORT

*Gray Code
&
Customized Fitness Function
Implementations in MATLAB*

TABLE OF CONTENTS

1. Introduction	2
2. Gray Code	2
Figure 1: Bit-by-bit derivation of Gray Code from binary representation 1011, illustrating the XOR-based conversion rule applied at each bit position	2
Figure 2: MATLAB implementation of Gray Code	2
Figure 3: Gray Code output sample in MATLAB	3
3. Customized Fitness Function	3
3.1.Fitness Function Implementation	3
Figure 4: MATLAB implementation of Customized Fitness Function including population initialization, fitness evaluation and evolutionary operators (View more in GitHub)	4
3.2.Results and Technical Interpretation	4
Figure 5: GA convergence plot showing the improvement of the best fitness value over generations toward the optimal solution	5
4. Conclusion	5



1. Introduction

This report is submitted as a midterm project for the Genetic Algorithms course. It covers two fundamental topics addressed in the scope of the course: the design and implementation of fitness function, and the representation of solutions using Gray Code encoding. Both implementations are carried out in MATLAB. The report includes code explanations, conversion examples and program outputs for each topic.

2. Gray Code

Gray Code is a binary numeral system where two successive values differ in only one bit. Unlike standard binary encoding where multiple bits can change simultaneously between consecutive numbers, Gray Code ensures a single-bit transition at each step. This property makes it particularly useful in genetic algorithms where it reduces the risk of large jumps in the solution space during mutation and crossover operations.

Conversion Rule (Binary to Gray Code): As shown in [Figure 1](#), each Gray Code bit is derived by applying the XOR operation between the current binary bit and the previous one. The Most Significant Bit (MSB) is copied directly.

```

Binary:    1  0  1  1
           |  |  |  |
G1 = B1          -> 1 (MSB)
G2 = B1 XOR B2   -> 1 XOR 0 = 1
G3 = B2 XOR B3   -> 0 XOR 1 = 1
G4 = B3 XOR B4   -> 1 XOR 1 = 0

Gray Code: 1  1  1  0

```

Figure 1: Bit-by-bit derivation of Gray Code from binary representation 1011, illustrating the XOR-based conversion rule applied at each bit position

The following script ([Figure 2](#)) generates a 4-bit Gray Code table for all decimal values from 0 to 15. The conversion is performed in a single line using MATLAB's built-in *bitxor* and *bitshift* functions.

```

1  %% Gray Code
2  clc; clear;
3
4  n_bits = 4;
5
6  fprintf('Decimal  Binary  GrayCode\n');
7  fprintf('-----\n');
8
9  for d = 0 : 2^n_bits - 1
10     gray = bitxor(d, bitshift(d, -1)); % one line gray code
11     fprintf(' %2d      %s      %s\n', d, dec2bin(d, n_bits), dec2bin(gray, n_bits));
12 end

```

Figure 2: MATLAB implementation of Gray Code

Command Window		
Decimal	Binary	GrayCode
0	0000	0000
1	0001	0001
2	0010	0011
3	0011	0010
4	0100	0110
5	0101	0111
6	0110	0101
7	0111	0100
8	1000	1100
9	1001	1101
10	1010	1111
11	1011	1110
12	1100	1010
13	1101	1011
14	1110	1001
15	1111	1000

The core formula $\text{bitxor}(d, \text{bitshift}(d, -1))$ works as follows: shifting d one bit to the right aligns each bit with its predecessor and applying XOR produces the Gray Code equivalent in a single operation — a direct implementation of the $G(i) = B(i) \text{ XOR } B(i-1)$ rule.

Figure 3: Gray Code output sample in MATLAB

3. Customized Fitness Function

In this part of the midterm, a simplified customized fitness function is designed using the genetic algorithm infrastructure provided during the lectures (20 April). Unlike the more complex examples discussed in class, this implementation focuses on a basic and easily interpretable optimization problem making it suitable for the midterm stage of the course. The objective of the fitness function is to maximize the following mathematical expression:

$$f(x, y) = \exp(-(x^2 + y^2))$$

This function has a known global maximum value of 1 which occurs at $x = 0$ and $y = 0$. Therefore, the goal of the genetic algorithm is to find parameter values that approach this optimal point.

3.1. Fitness Function Implementation

Each individual in the population represents two parameters, x and y , encoded in binary form. The genetic algorithm follows the standard procedure discussed in the lectures; binary chromosomes are first generated using the `genhavr` function, then decoded into integer values using the `createchr` function, and finally mapped into real-valued parameters within the range $[-5, 5]$ using the `gendeg` function.

In the updated implementation, the customized fitness function is integrated into a complete genetic algorithm loop. At each iteration (generation), the population is evaluated using the fitness function defined as $f(x, y) = \exp(-(x^2 + y^2))$, where higher values indicate better solutions.

After evaluating all individuals, fitness values are sorted and the best individual is selected. An elitism strategy is applied by preserving the best chromosome within the population to ensure that high-quality solutions are not lost between generations.

The genetic algorithm then applies evolutionary operators to generate a new population. Specifically, crossover is used to recombine selected chromosomes and explore new candidate solutions, while mutation introduces random variations to maintain diversity within the population and prevent premature convergence.

```

main.m x fitness_network.m x visualize_results.m x fitness_simple.m x +
1 %% Simple Fitness Function for midterm project (clean version of fitnesscalc1.m)
2 % Objective: maximize f(x,y) = exp(-(x^2 + y^2))
3 clc; clear;
4 addpath('..\lecture-codes')
5 %% GA Parameters
6 pop = genhav(30, 2, 8); % initial population
7 maxGen = 200; % number of generations
8 mutationRate = 0.02;
9 %% Main GA Loop
10 for epoch = 1:maxGen
11     % Decode chromosome -> real values
12     [paramvalue, ~] = createchr(pop, 2, 8);
13     genvalue = gendeg(paramvalue, [-5 -5], [5 5], 8);
14
15     % Fitness evaluation
16     [bstscore, score, bestchr, prmbest, pop] = fitnesscalc_simple(genvalue, pop);
17
18     % Store best fitness
19     bestFitness(epoch) = bstscore;
20
21     % GA operators
22     pop1 = crossover(pop, bestchr);
23     pop2 = mutation1(pop1, bestchr, mutationRate);
24
25     % Update population
26     pop = pop2;
27 end
28 %% Final result
29 fprintf('\nFINAL RESULT:\n');
30 fprintf('Best fitness value = %.4f\n', bstscore);
31 fprintf('Best x = %.4f, Best y = %.4f\n', prmbest(1), prmbest(2));
32
33 %% Plot convergence
34 figure;
35 plot(bestFitness, 'LineWidth', 2);
36 xlabel('Generation');
37 ylabel('Best Fitness');
38 title('GA Convergence');
39 grid on;

```

Figure 4: MATLAB implementation of Customized Fitness Function including population initialization, fitness evaluation and evolutionary operators ([View more in GitHub](#))

This process is repeated over multiple generations within an iterative loop, allowing the population to evolve gradually toward the optimal solution. The best fitness value is recorded at each generation, enabling convergence analysis and performance evaluation of the algorithm.

Figure 4 shows the MATLAB implementation of the updated genetic algorithm structure, including population initialization, fitness evaluation, and evolutionary operations across generations.

3.2. Results and Technical Interpretation

The execution of the fitness function produced the following result:

- Best fitness value: 0.9992
- Best parameters: $x = 0.0196, y = 0.0196$

Unlike the initial implementation, the fitness evaluation in this version is performed within an iterative genetic algorithm loop including crossover and mutation operations. This allows the population to evolve over multiple generations and converge toward the global optimum.

Substituting the obtained parameter values into the fitness function:

$$x^2 + y^2 = (0.0196)^2 + (0.0196)^2 \approx 0.000768$$

$$f(x, y) = \exp(-0.000768) \approx 0.9992$$

which closely matches the reported best fitness value. This confirms that the fitness function, decoding process and evaluation mechanism are mathematically consistent and correctly implemented.

The result is now very close to the global maximum value of 1 indicating that the genetic algorithm successfully converged to the optimal solution. This behavior is achieved by introducing an iterative process in which selection, crossover and mutation operators are applied across multiple generations.

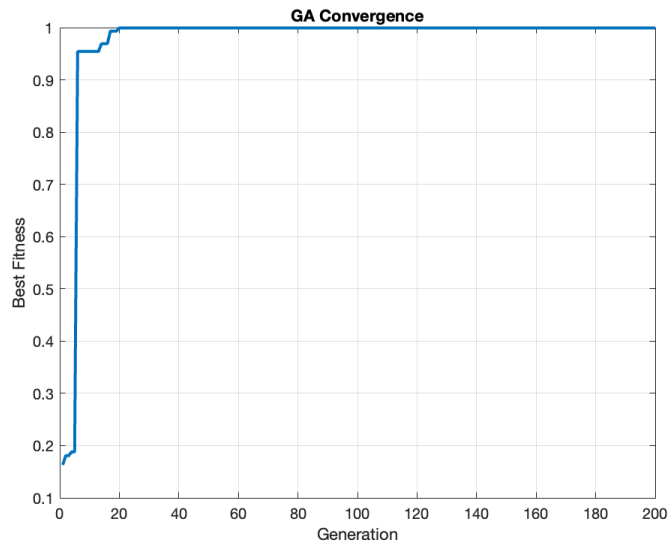


Figure 5: GA convergence plot showing the improvement of the best fitness value over generations toward the optimal solution

The convergence plot (Figure 5) further supports this observation, showing a rapid increase in fitness values during early generations followed by stabilization near the optimal value. This demonstrates that the genetic algorithm effectively improves candidate solutions and reaches a stable optimum.

Overall, the updated model accurately reflects the full genetic algorithm workflow and successfully demonstrates the optimization capability of evolutionary processes.

4. Conclusion

In this midterm project, two core concepts of genetic algorithms were studied and implemented using MATLAB. The first part focused on solution representation through Gray Code encoding where the conversion process was explained and demonstrated with sample inputs and outputs.

The second part of the project examined the design of a customized fitness function. A simplified, problem-specific fitness function was developed to demonstrate how objective functions can be designed and integrated into a genetic algorithm framework. Due to its simplicity, the optimization behavior can be easily analyzed and explained, making it suitable for the midterm stage. The results obtained from the fitness evaluation confirm that the defined fitness function is correctly implemented and that the decoding and evaluation mechanisms behave as expected.

This project represents the completion of the midterm stage of the overall study. In the final stage, the fitness function and genetic algorithm structure will be further improved by introducing iterative evolution, convergence analysis and additional constraints. Overall, this project helped reinforce the theoretical concepts of genetic algorithms through practical MATLAB implementations and provided a solid baseline for more advanced optimization tasks in the final project.